



NILS HARTMANN

<https://nilshartmann.net>

Hands-on Spring Modulith

Eine praktische Einführung

ENTWICKLER.DE | 24. FEBRUAR 2026 | [HTTPS://SPRING.SCHULE](https://spring.schule)

NILS HARTMANN

nils@nilshartmann.net

Freiberuflicher Software-Entwickler, –Architekt, Coach, Trainer

Java, Spring, GraphQL, React, TypeScript

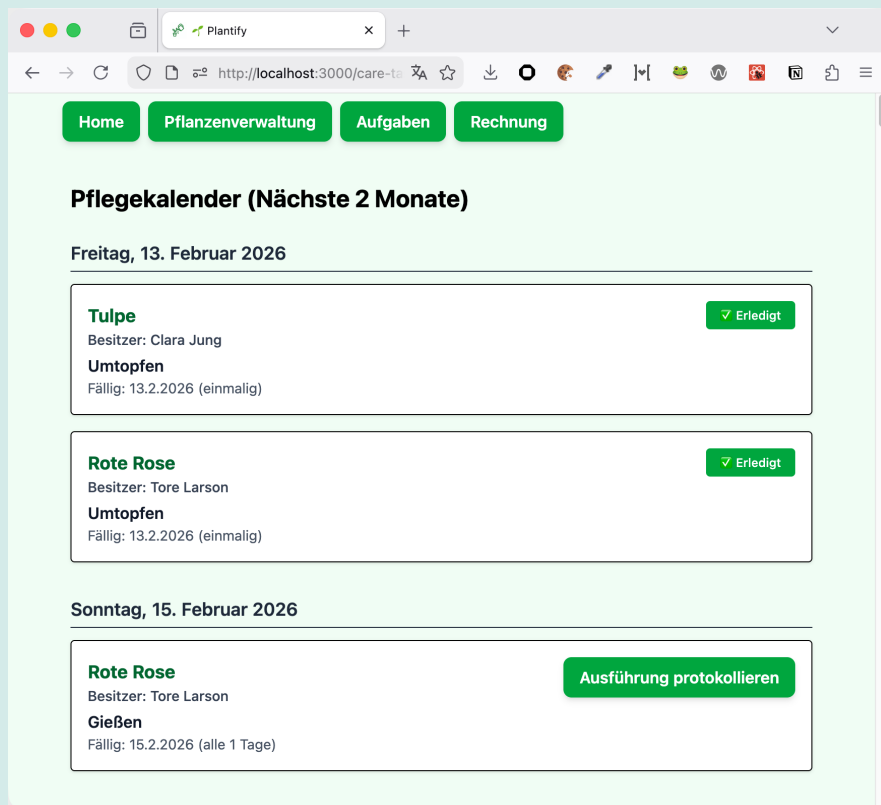


<https://react.schule/video-kurs-react>



<https://reactbuch.de>

[HTTPS://NILSHARTMANN.NET](https://nilshartmann.net)



BEISPIEL

- Wie gliedern wir die Anwendung?

- Wie gliedern wir die Anwendung?

1

nh.demo.plantify

- © Billing
- © BillingController
- © BillingDto
- © BillingRepository
- © BillingService
- © Plant
- © PlantController
- © PlantDto
- © PlantRepository
- © PlantService

• Wie gliedern wir die Anwendung?

1

- nh.demo.plantify
 - Billing
 - BillingController
 - BillingDto
 - BillingRepository
 - BillingService
 - Plant
 - PlantController
 - PlantDto
 - PlantRepository
 - PlantService

2

- nh.demo.plantify
 - domain
 - Billing
 - BillingRepository
 - BillingService
 - Plant
 - PlantRepository
 - PlantService
 - web
 - BillingController
 - BillingDto
 - PlantController
 - PlantDto

• Wie gliedern wir die Anwendung?

1

- nh.demo.plantify
 - Billing
 - BillingController
 - BillingDto
 - BillingRepository
 - BillingService
 - Plant
 - PlantController
 - PlantDto
 - PlantRepository
 - PlantService

2

- nh.demo.plantify
 - domain
 - Billing
 - BillingRepository
 - BillingService
 - Plant
 - PlantRepository
 - PlantService
 - web
 - BillingController
 - BillingDto
 - PlantController
 - PlantDto

3

- nh.demo.plantify
 - billing
 - Billing
 - BillingController
 - BillingDto
 - BillingRepository
 - BillingService
 - plant
 - Plant
 - PlantController
 - PlantDto
 - PlantRepository
 - PlantService

ARCHITEKTUR VON PLANTIFY

• Wie gliedern wir die Anwendung?

1

- nh.demo.plantify
 - Billing
 - BillingController
 - BillingDto
 - BillingRepository
 - BillingService
 - Plant
 - PlantController
 - PlantDto
 - PlantRepository
 - PlantService

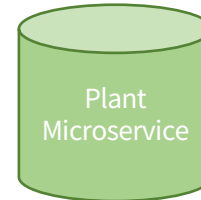
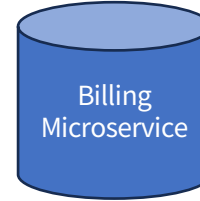
2

- nh.demo.plantify
 - domain
 - Billing
 - BillingRepository
 - BillingService
 - Plant
 - PlantRepository
 - PlantService
 - web
 - BillingController
 - BillingDto
 - PlantController
 - PlantDto

3

- nh.demo.plantify
 - billing
 - Billing
 - BillingController
 - BillingDto
 - BillingRepository
 - BillingService
 - plant
 - Plant
 - PlantController
 - PlantDto
 - PlantRepository
 - PlantService

4



ARCHITEKTUR VON PLANTIFY

• Wie gliedern wir die Anwendung?

1

- nh.demo.plantify
 - Billing
 - BillingController
 - BillingDto
 - BillingRepository
 - BillingService
 - Plant
 - PlantController
 - PlantDto
 - PlantRepository
 - PlantService

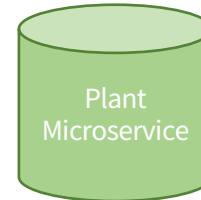
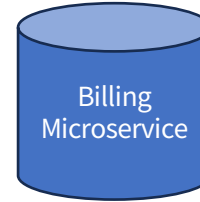
2

- nh.demo.plantify
 - domain
 - Billing
 - BillingRepository
 - BillingService
 - Plant
 - PlantRepository
 - PlantService
 - web
 - BillingController
 - BillingDto
 - PlantController
 - PlantDto

3

- nh.demo.plantify
 - billing
 - Billing
 - BillingController
 - BillingDto
 - BillingRepository
 - BillingService
 - plant
 - Plant
 - PlantController
 - PlantDto
 - PlantRepository
 - PlantService

4



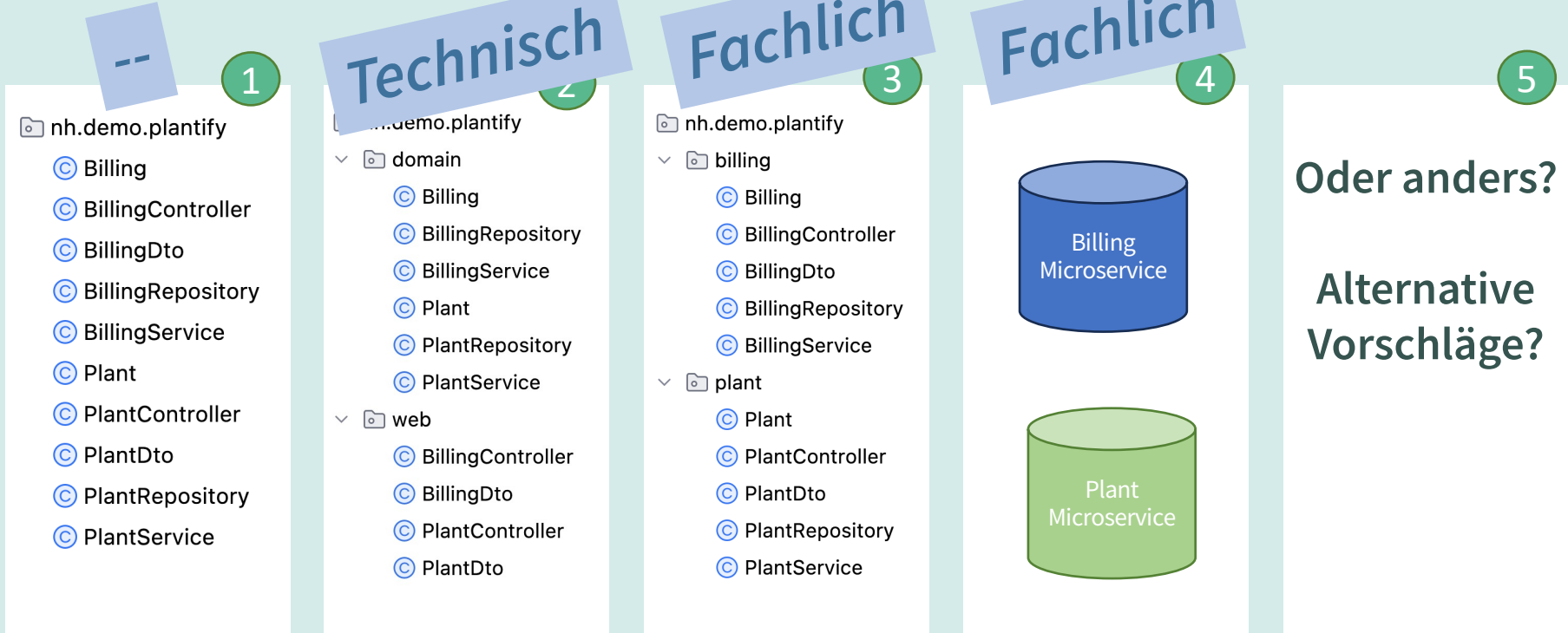
5

Oder anders?

Alternative
Vorschläge?

ARCHITEKTUR VON PLANTIFY

• Wie gliedern wir die Anwendung?



- Fachliche „Slices“

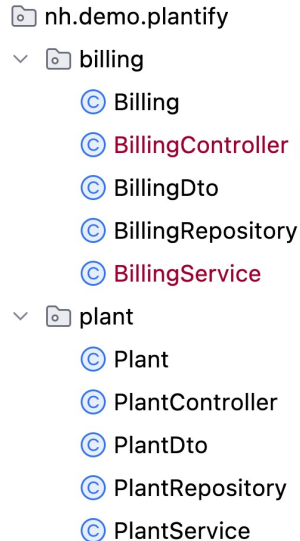
Fachlich

```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
├── plant
│   ├── Plant
│   ├── PlantController
│   ├── PlantDto
│   ├── PlantRepository
│   └── PlantService
```

- Fachlichkeit ist an einer Stelle konzentriert
- („Jetzt wächst zusammen, was zusammen gehört“)

- Fachliche „Slices“

Fachlich



```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
├── plant
│   ├── Plant
│   ├── PlantController
│   ├── PlantDto
│   ├── PlantRepository
│   └── PlantService
```

- Fachlichkeit ist an einer Stelle konzentriert
- („Jetzt wächst zusammen, was zusammen gehört“)
- **Änderungen** müssen nur an **einer Stelle** gemacht werden

- Fachliche Slices

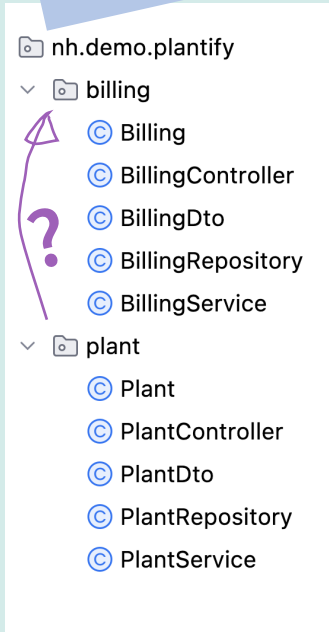
Fachlich

```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
├── plant
│   ├── Plant
│   ├── PlantController
│   ├── PlantDto
│   ├── PlantRepository
│   └── PlantService
```

- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen

- Fachliche Slices

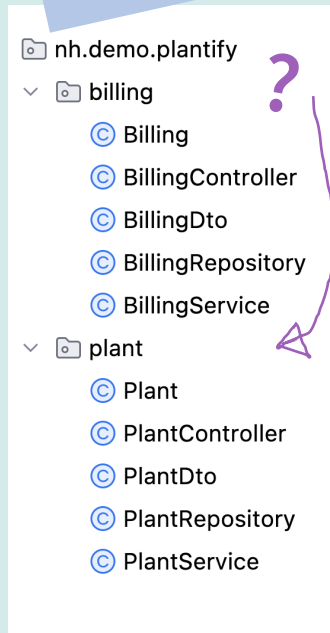
Fachlich



- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sehen Abhängigkeiten aus?

- Fachliche Slices

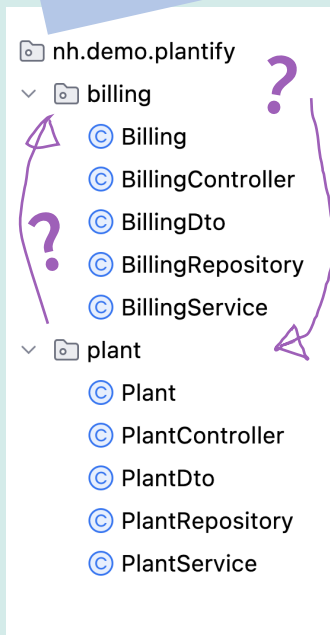
Fachlich



- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sehen Abhängigkeiten aus?
- Oder so?

- Fachliche Slices

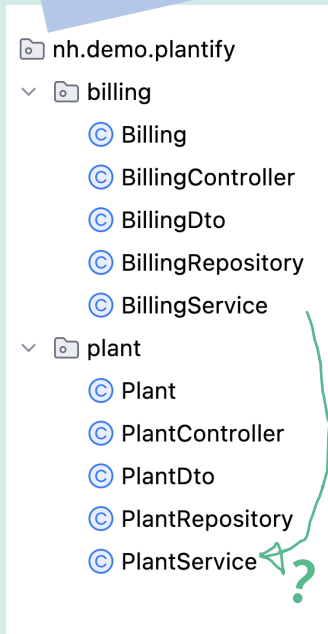
Fachlich



- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sehen Abhängigkeiten aus?
- Oder so?
- Mal so, mal so 🤔

• Schnittstellen

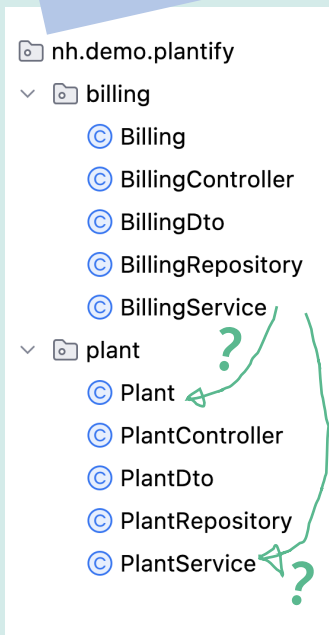
Fachlich



- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die **Schnittstelle** von „plant“ aus?

• Schnittstellen

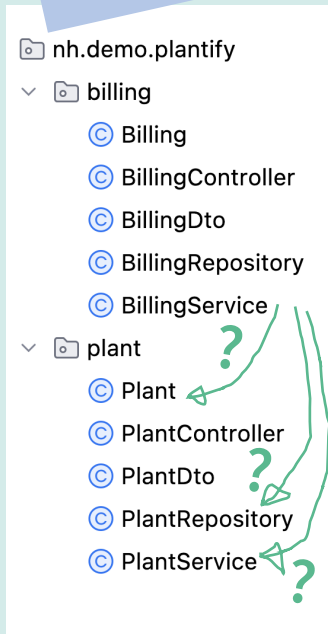
Fachlich



- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔

• Schnittstellen

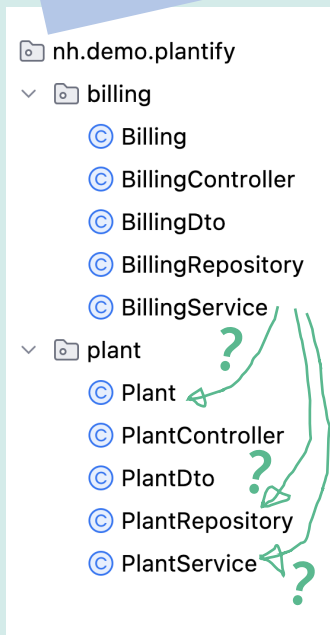
Fachlich



- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔
- 🤔 🤔

• Schnittstellen

Fachlich

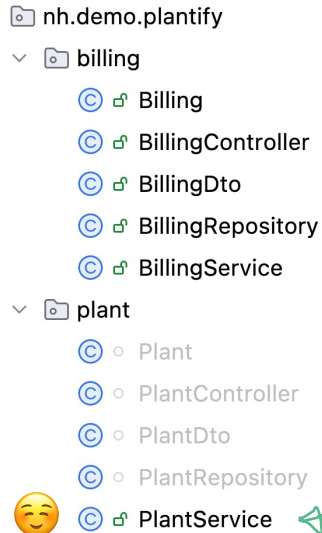


- Problem: Fachliche Slices existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die Schnittstelle von „plant“ aus?
- 🤔
- 🤔 🤔
- (Warum) ist das überhaupt relevant?

ARCHITEKTUR VON PLANTIFY

- Schnittstellen per Java Bord-Mittel

Fachlich



- Sichtbarkeit von Klassen einschränken

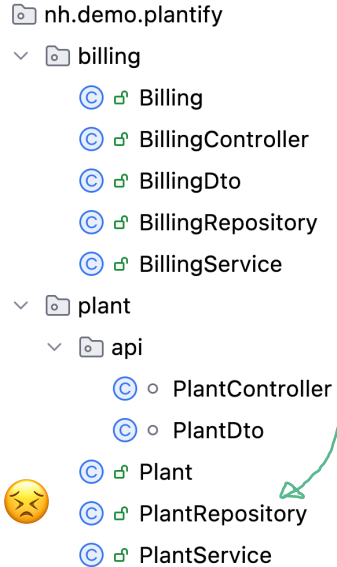
```
package nh.demo.plantify.plant;  
  
// Öffentlich  
public class PlantService {  
  
    // ...  
  
}
```

```
package nh.demo.plantify.plant;  
  
// Nur Plant-intern  
class PlantRepository {  
  
    // ...  
  
}
```

ARCHITEKTUR VON PLANTIFY

• Schnittstellen per Java Bord-Mittel

Fachlich



• Sichtbarkeit von Klassen einschränken

```
package nh.demo.plantify.plant;  
  
// Öffentlich  
public class PlantService {  
  
    // ...  
  
}
```



```
package nh.demo.plantify.plant;  
  
// Nur Plant intern  
public class PlantRepository {  
  
    // ...  
  
}
```

ARCHITEKTUR VON PLANTIFY

- Schnittstellen per Java Bord-Mittel

Fachlich

Weitere Möglichkeiten? 🤔

- Sichtbarkeit von Klassen etc.

nh.demo.plantify

▼ billing

🔒 Billing

🔒 BillingC

▼ plantify

▼ api

🔒 PlantController

🔒 PlantDto



🔒 Plant

🔒 PlantRepository

🔒 PlantService

... PlantService {

// ...

}



// ~~Nur Plant intern~~

public class PlantRepository {

// ...

}



- Fachliche Slices

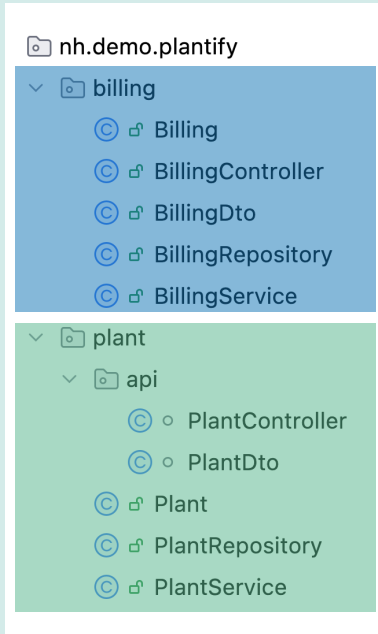
Was fehlt?

```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
├── plant
│   └── api
│       ├── PlantController
│       ├── PlantDto
│       ├── Plant
│       ├── PlantRepository
│       └── PlantService
```

- Fachliche Slices

Was fehlt?

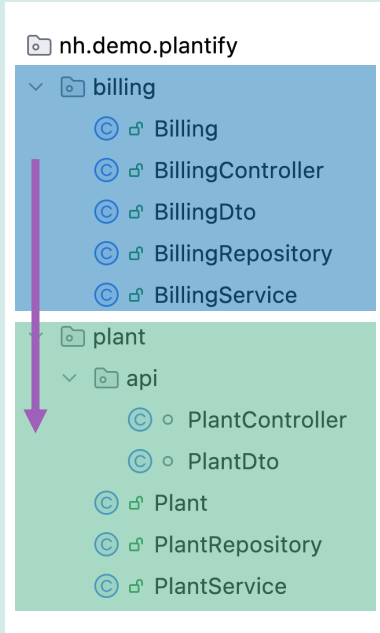
- „Echte“ Slices
 - Mehr als Klassen oder Packages



- Fachliche Slices

Was fehlt?

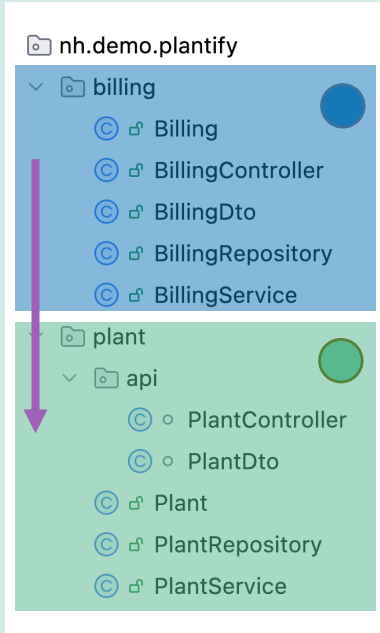
- „Echte“ Slices
 - Mehr als Klassen oder Packages
- Klare Abhängigkeiten
 - Zwischen den Slices



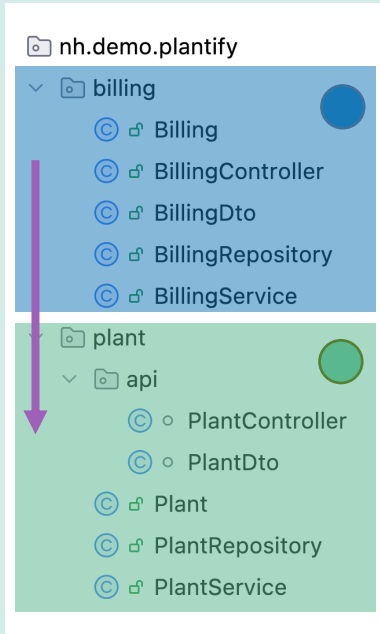
- Fachliche Slices

Was fehlt?

- „Echte“ Slices
 - Mehr als Klassen oder Packages
- Klare Abhängigkeiten
 - Zwischen den Slices
- Klare Schnittstellen
 - Zwischen den Slices

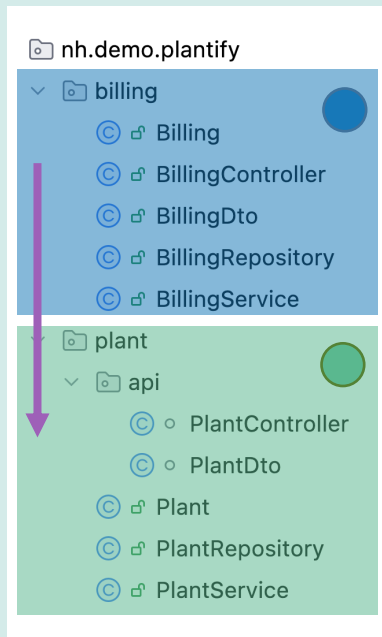


- Spring Modulith



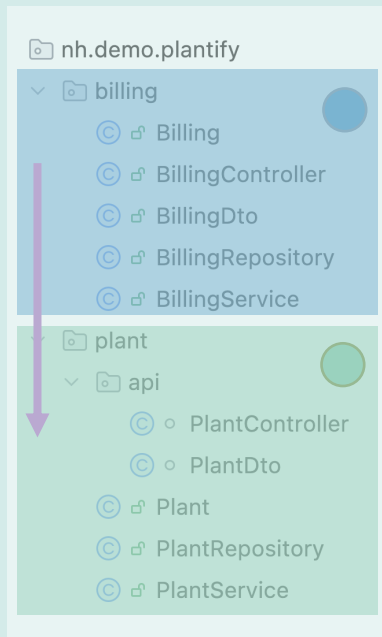
- Application Module
 - Per Default: Packages
 - Definierte Abhängigkeiten
 - Explizite Schnittstellen

- Spring Modulith



- Application Module
 - Per Default: Packages
 - Definierte Abhängigkeiten
 - Explizite Schnittstellen
- Isolierte Tests
- Kommunikation per Events

- Spring Modulith



- Application Module

- Per Default: D... ges
- ...keiten
- ... Schnittstellen

- Isolierte Tests

- Kommunikation per Events

NILS HARTMANN

<https://nilshartmann.net>

Vielen Dank!

Code: <https://spring.schule/entwicklerde-spring-modulith>

Fragen und Kontakt:

nils@nilshartmann.net

<https://nilshartmann.net/kontakt>

